

STEP 1 — Install & Import libraries

```
!pip install torch torchvision scikit-learn nltk
```

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
```

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
```

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
```

```
nltk.download('punkt')
nltk.download('stopwords')
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.25.0+cpu)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.25.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.3)
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
True
```

STEP 2 — Load Dataset

```
categories = ['sci.space', 'rec.sport.hockey'] # binary classification
```

```
data = fetch_20newsgroups(subset='all', categories=categories,
                          remove=('headers', 'footers', 'quotes'))
```

```
texts = data.data
labels = data.target
```

```
print("Sample text:\n", texts[0])
print("Label:", labels[0])
```

Sample text:

Well, I guess I'm left wondering just who all the 'light fascists' think *they* are. Yes, I understand the issues. I don't even particularly care for the idea. But am I the only one that finds the sort of overreaction above just a *little* questionable? You must find things like the Moon *really* obnoxious in their pollution.

A few questions for those frothing at the mouth to ask themselves:

1) How long is this thing supposed to stay up? Sounds like it would have a *huge* drag area, not a lot of mass, and be in a fairly low orbit.

2) Just what orbital parameters are we talking about here? What real impact are we talking about, really? How many optical astronomers are *really* going to be impacted?

3) Which is more important; adding a few extra days of 'seeing' for (very few) optical astronomers or getting the data the sensors are supposed to return along with the data for large inflatables (and the potential there for an inflatable space station)? The choice would seem to be one or the other, since the advertising is being used to help fund this thing.

4) If your answer to 3) above was "the astronomers", then feel free to come up with some other way to fund the (to my mind) more important research data that would be gained by this WITHOUT SPENDING ANY MORE OF MY MONEY TO DO IT. In other words, put up or shut up.

Probably so. I'm just not sure we agree about who the 'stupid' are.

Yeah, me too.

--
 "Insisting on perfect safety is for people who don't have the balls to live in the real world." -- Mary Shafer, NASA Ames Dryden
 Label: 1

STEP 3 — Preprocessing

```
import nltk
nltk.download('punkt_tab')

stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = text.lower()
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word.isalpha()] # remove punctuation
    tokens = [word for word in tokens if word not in stop_words]
    return tokens

processed_texts = [preprocess(t) for t in texts]

print(processed_texts[0][:20])
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
['well', 'guess', 'left', 'wondering', 'fascists', 'think', 'yes', 'understand', 'issues', 'even', 'particularly', 'care', 'idea
```

STEP 4 — Vocabulary + Encoding

```
vocab = {}
for tokens in processed_texts:
    for word in tokens:
        if word not in vocab:
            vocab[word] = len(vocab) + 1 # start from 1

max_len = 100

def encode(tokens):
    seq = [vocab[word] for word in tokens if word in vocab]
    if len(seq) < max_len:
        seq += [0] * (max_len - len(seq)) # padding
    else:
        seq = seq[:max_len]
    return seq

encoded_texts = np.array([encode(t) for t in processed_texts])
```

```
labels = np.array(labels)

print(encoded_texts.shape)

(1986, 100)
```

STEP 5— Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    encoded_texts, labels, test_size=0.2, random_state=42
)
```

STEP 6 — PyTorch Dataset Class

```
class TextDataset(torch.utils.data.Dataset):
    def __init__(self, X, y):
        self.X = torch.LongTensor(X)
        self.y = torch.LongTensor(y)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

train_dataset = TextDataset(X_train, y_train)
test_dataset = TextDataset(X_test, y_test)

train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=32)
```

STEP 7 — Build 1D CNN Model

```
class CNNModel(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_classes):
        super(CNNModel, self).__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim)

        self.conv = nn.Conv1d(in_channels=embed_dim,
                               out_channels=100,
                               kernel_size=3)

        self.pool = nn.MaxPool1d(kernel_size=2)
        self.fc = nn.Linear(100 * 49, num_classes) # depends on max_len

        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.embedding(x) # (batch, seq_len, embed_dim)
        x = x.permute(0, 2, 1) # (batch, embed_dim, seq_len)

        x = self.conv(x)
        x = self.relu(x)
        x = self.pool(x)

        x = x.view(x.size(0), -1)
        x = self.fc(x)

        return x

vocab_size = len(vocab) + 1
model = CNNModel(vocab_size, embed_dim=50, num_classes=2)
```

STEP 8 — Training

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

epochs = 5
```

```

for epoch in range(epochs):
    model.train()
    total_loss = 0

    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()

        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {total_loss:.4f}")

```

```

Epoch 1, Loss: 40.3774
Epoch 2, Loss: 25.6584
Epoch 3, Loss: 16.1567
Epoch 4, Loss: 9.5162
Epoch 5, Loss: 6.2646

```

STEP 9 — Evaluation

```

model.eval()

all_preds = []
all_labels = []

with torch.no_grad():
    for X_batch, y_batch in test_loader:
        outputs = model(X_batch)
        _, preds = torch.max(outputs, 1)

        all_preds.extend(preds.numpy())
        all_labels.extend(y_batch.numpy())

print("\nClassification Report:\n")
print(classification_report(all_labels, all_preds))

```

Classification Report:

	precision	recall	f1-score	support
0	0.85	0.87	0.86	202
1	0.86	0.84	0.85	196
accuracy			0.85	398
macro avg	0.85	0.85	0.85	398
weighted avg	0.85	0.85	0.85	398

STEP 10 — Confusion Matrix

```

cm = confusion_matrix(all_labels, all_preds)
print("\nConfusion Matrix:\n")
print(cm)

```

Confusion Matrix:

```

[[175  27]
 [ 32 164]]

```

STEP 10 — Naive Bayes Baseline (for Comparison)

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

clean_texts = [" ".join(t) for t in processed_texts]

X_train_nb, X_test_nb, y_train_nb, y_test_nb = train_test_split(
    clean_texts, labels, test_size=0.2, random_state=42)

```

```
clean_texts, labels, test_size=0.2, random_state=42
)

vectorizer = TfidfVectorizer(max_features=5000)
X_train_nb = vectorizer.fit_transform(X_train_nb)
X_test_nb = vectorizer.transform(X_test_nb)

nb_model = MultinomialNB()
nb_model.fit(X_train_nb, y_train_nb)

nb_preds = nb_model.predict(X_test_nb)

print("\nNaive Bayes Report:\n")
print(classification_report(y_test_nb, nb_preds))
```

Naive Bayes Report:

	precision	recall	f1-score	support
0	0.93	0.98	0.95	202
1	0.97	0.92	0.95	196
accuracy			0.95	398
macro avg	0.95	0.95	0.95	398
weighted avg	0.95	0.95	0.95	398